BTK AKADEMİ

İLERİ SEVİYE PYTHON PROGRAMLAMA DİLİ

İleri Seviye Fonksiyonlar

EBRU AYDOĞAN

İç İçe (Nested) Fonksiyonlar

- Neden İç İçe (Nested) Fonksiyon Kullanıyoruz?
  - **Kapsülleme (Encapsulation):** Bir fonksiyonu sadece başka bir fonksiyonun içinde kullanmak üzere saklamak (Gizlilik).
  - **Kod Tekrarını Önlemek:** Ana fonksiyonun içinde defalarca tekrarlanan küçük ama spesifik işlemleri bir alt fonksiyona devretmek.
  - **Closure ve Decorator Temeli:** Python'ın en ileri özelliklerinden olan sarmalayıcıları (decorators) oluşturabilmek için bu yapı zorunludur.
  - **Bellek Yönetimi:** İçteki fonksiyon, sadece dıştaki fonksiyon çalıştığı sürece var olur.

İç İçe (Nested) Fonksiyonlar

- İç İçe Fonksiyon Mantığı
  - **Kural:** İçteki fonksiyon (inner), dıştaki fonksiyonun (outer) değişkenlerine erişebilir; ancak dıştaki fonksiyon içtekinin değişkenlerini göremez.
  - Scope (Kapsam) Hiyerarşisi:
    - 1. Local (İç fonksiyonun içi)
    - 2. Enclosing (Dış fonksiyonun içi)
    - 3. Global (Kodun genel alanı)

İç İçe (Nested) Fonksiyonlar

- Örnek: Temel bir selamlama

```
def dis_fonksiyon(mesaj):
    # Dış fonksiyonun değişkenini kullanıyoruz
    def ic_fonksiyon():
        print(f"Sistemden bir mesajınız var: {mesaj}")

    ic_fonksiyon()

dis_fonksiyon("Eğitim Başladı!")
```

BTK AKADEMİ

İç İçe (Nested) Fonksiyonlar

- Örnek: Matematiksel Kapsülleme

```
def hesaplama(sayi1, sayi2):
    def topla():
        return sayi1 + sayi2

    def carp():
        return sayi1 * sayi2

    # Sadece sonuçları döndürüyoruz, fonksiyonlar içeride gizli kalıyor
    return f"Toplam: {topla()}, Çarpım: {carp()}"

print(hesaplama(10, 5))
```

BTK AKADEMİ

İç İçe (Nested) Fonksiyonlar

- Örnek: Faktöriyel Kontrolü

```python
def faktoriyel_hesapla(n):
    # Veri doğrulamayı dışarıda yapıyoruz
    if not isinstance(n, int) or n < 0:
        return "Hatalı giriş! Lütfen pozitif tam sayı girin."

    # Gerçek hesaplamayı içerideki fonksiyona yaptırıyoruz
    def hesapla(num):
        if num <= 1:
            return 1
        return num * hesapla(num - 1)

    return hesapla(n)

print(faktoriyel_hesapla(5)) # 120
print(faktoriyel_hesapla(-3)) # Hata mesajı
```

BTK AKADEMİ

Fonksiyondan Fonksiyon Döndürme (Closure)

- Fonksiyon Döndürmek Nedir?
  - **Mantık:** Bir fonksiyon çalıştığında bize int, str veya list döndürebildiği gibi, çalışmaya hazır **başka bir fonksiyon** da döndürebilir.
  - Neden Kullanılır?
    - Kişiselleştirilmiş fonksiyonlar üretmek için
    - Dış fonksiyon bittikten sonra bile onun içindeki değişkenlerin hatırlanmasını sağlamak.
  - **Sözdizimi:** return ic_fonksiyon (Parantez kullanmadan ismiyle döndürülür!)

BTK AKADEMİ

Fonksiyondan Fonksiyon Döndürme (Closure)

- Örnek: Temel üs alma

```
def us_al(ust):
    def hesapla(taban):
        return taban ** ust
    return hesapla # Fonksiyonun kendisini döndürüyoruz

# Fabrikayı kullanma:
kare_al = us_al(2) # Artık kare_al bir fonksiyondur ve ust=2'yi hatırlar
kup_al = us_al(3)  # Artık kup_al bir fonksiyondur ve ust=3'ü hatırlar

print(kare_al(5)) # 25
print(kup_al(2))  # 8
```

Fonksiyondan Fonksiyon Döndürme (Closure)

- Örnek: Kişiye özel selamlama

```
def yetki_sorgula(rol):
    def mesaj_ver(isim):
        return f"{rol} yetkisiyle: Merhaba {isim}"
    return mesaj_ver

admin_selami = yetki_sorgula("Yönetici")
user_selami = yetki_sorgula("Standart Kullanıcı")

print(admin_selami("Ebru")) # Yönetici yetkisiyle: Merhaba Ebru
print(user_selami("Ali"))   # Standart Kullanıcı yetkisiyle: Merhaba Ali
```

Fonksiyondan Fonksiyon Döndürme (Closure)

- Örnek: Dinamik yetki kontrolü

```python
def yetki_sorgulama(sayfa):
    def inner(role):
        if role == "Admin":
            return f"{role} rolü {sayfa} sayfasına ulaşabilir."
        else:
            return f"{role} rolü {sayfa} sayfasına ulaşamaz."

    return inner

# Sayfaya özel kontrol fonksiyonu üretiyoruz
yetki = yetki_sorgulama("Ürün güncelleme")

# Şimdi bu fonksiyonu farklı rollerle test ediyoruz
print(yetki("User"))   # User rolü Ürün güncelleme sayfasına ulaşamaz.
print(yetki("Admin"))  # Admin rolü Ürün güncelleme sayfasına ulaşabilir.
```

BTK AKADEMİ

Fonksiyondan Fonksiyon Döndürme (Closure)

- Örnek: Basit Bir Sayaç

```
def sayac_olustur():
    sayi = 0
    def artir():
        nonlocal sayi # Dış fonksiyondaki 'sayi'yı değiştirmek için
        sayi += 1
        return sayi
    return artir

sayac = sayac_olustur()

print(sayac()) # 1
print(sayac()) # 2
print(sayac()) # 3
```

BTK AKADEMİ

Fonksiyonları Parametre Olarak Gönderme

- Fonksiyon Bir Değişken Gibidir
  - **Ana Fikir:** Python'da fonksiyonlar "First-Class Objects" (Birinci Sınıf Nesneler) olarak kabul edilir.
  - **Bu Ne Demek?** Bir fonksiyonu bir değişkene atayabilir, bir listenin içine koyabilir veya başka bir fonksiyona parametre olarak gönderebilirsiniz.
  - **Neden Kullanılır?** Kodun modülerliğini artırmak için. Bir fonksiyonun "ne yapacağını" değil, "nasıl bir işlemle yapacağını" dışarıdan belirlemiş olursunuz.

Fonksiyonları Parametre Olarak Gönderme

- Örnek: Temel işlem gönderimi

```
def topla(a, b):
    return a + b

def carp(a, b):
    return a * b

def islem_yap(fnc, x, y):
    # fnc parametresi aslında bir fonksiyon!
    return fnc(x, y)

print(islem_yap(topla, 5, 3)) # Çıktı: 8
print(islem_yap(carp, 5, 3))  # Çıktı: 15
```

BTK AKADEMİ

Fonksiyonları Parametre Olarak Gönderme

- Örnek: Liste Elemanlarını Fonksiyonla İşlemek

```
def kare_al(n):
    return n ** 2

def kup_al(n):
    return n ** 3

def listeyi_isle(fnc, liste):
    sonuc = []
    for eleman in liste:
        sonuc.append(fnc(eleman))
    return sonuc

sayilar = [1, 2, 3, 4]
print(listeyi_isle(kare_al, sayilar)) # [1, 4, 9, 16]
print(listeyi_isle(kup_al, sayilar))  # [1, 8, 27, 64]
```

BTK AKADEMİ

Fonksiyonları Parametre Olarak Gönderme

- Örnek: Mesaj Formatlama

```
def buyuk_harf(metin):
    return metin.upper()

def kucuk_harf(metin):
    return metin.lower()

def yazdir(fnc, mesaj):
    print("Sistem Mesajı:", fnc(mesaj))

yazdir(buyuk_harf, "merhaba btk") # Sistem Mesajı: MERHABA BTK
yazdir(kucuk_harf, "SELAM")       # Sistem Mesajı: selam
```

BTK AKADEMİ

Uygulama 1: Matematiksel Kapsülleme

- Görev: Bir islem_merkezi(sayi1, sayi2) fonksiyonu yazın.
- Detay: Bu fonksiyonun içinde toplama, cikarma ve bolme adında 3 iç fonksiyon olsun.
- Çıktı: Ana fonksiyon çağrıldığında bu 3 işlemi de yapıp sonuçları bir sözlük (dictionary) olarak döndürsün.
  - Örn: {"toplam": 15, "fark": 5, "bolum": 2}

BTK AKADEMİ

Uygulama 2: Kişiselleştirilmiş Logger

- Görev: Bir logger(seviye) fonksiyonu yazın. (Örn seviyeler: "INFO", "ERROR", "DEBUG")
- Detay: Bu fonksiyon geriye bir mesaj alan mesaj_yaz(metin) fonksiyonu döndürsün.
- Kullanım:

```
bilgi_log = logger("INFO")
hata_log = logger("ERROR")

bilgi_log("Sistem çalışıyor") # [INFO]: Sistem çalışıyor
hata_log("Bağlantı kesildi")  # [ERROR]: Bağlantı kesildi
```

BTK AKADEMİ

Uygulama 3: Esnek Hesaplayıcı

- Görev: hesap_makinesi(liste, islem_fonksiyonu) adında bir fonksiyon yazın.
- Detay:
  - 1. İki ayrı fonksiyon tanımlayın: kare_al ve kok_al.
  - 2. hesap_makinesi fonksiyonuna bir sayı listesi ve bu işlemlerden birini göndererek sonuçları liste olarak alın.

BTK AKADEMİ